RESIDÊNCIA EM TIC 29 — WEB 3.0

EduProtocol

Protocolo Descentralizado de Educação Web3

Unidade	1 - Capítulo 5
Atividade	Desenvolvimento de Protocolo Web3 Completo com Deploy em Testnet
Nível	Avançado 2 de 3
Aluno	Alanio Ferreira de Lima
Professor	Bruno Portes
Rede de Deploy	Sepolia Testnet (Ethereum)
Data	Abril de 2025
Projeto Git	https://github.com/Allanio007/eduprotocol?

Sumário

1. Definição do Problema 3
2. Arquitetura do Protocolo 3
3. Justificativa dos Padrões ERC 4
4. Implementação Técnica 6
4.1 Token ERC-20 — EduToken (EDU) 6
4.2 NFT ERC-721 — EduNFT (EduCertificate) 6
4.3 Contrato de Staking — EduStaking 6
4.4 Governança — EduDAO 8
5. Segurança 8
6. Integração com Oráculo Chainlink 9
7. Integração Web3 — ethers.js 10
8. Deploy em Testnet Sepolia 12
9. Relatório de Auditoria 13
10. Conclusão 14

1. Definição do Problema

O mercado de educação online movimenta bilhões de dólares anualmente, porém apresenta falhas estruturais que prejudicam diretamente os alunos. Plataformas centralizadas como Coursera, Udemy e similares detêm controle unilateral sobre certificados emitidos — podendo revogá-los, alterar critérios retroativamente ou simplesmente encerrar as operações, tornando inválidas as conquistas dos estudantes.

Além disso, não existe mecanismo transparente de incentivo ao engajamento contínuo, nem participação real dos alunos nas decisões da plataforma (seleção de cursos, parcerias com instrutores, política de preços).

O EduProtocol resolve esses problemas por meio de quatro pilares:

1. Imutabilidade: certificados emitidos como NFTs na blockchain não podem ser revogados ou adulterados por nenhuma parte centralizada.
2. Incentivo tokenizado: alunos recebem tokens EDU por engajamento e podem ampliá-los via staking com recompensas dinâmicas.
3. Governança descentralizada: detentores de EDU votam em propostas sobre o futuro da plataforma, eliminando decisões arbitrárias.
4. Transparência total: todas as transações, votos e recompensas são verificáveis publicamente na blockchain Ethereum.

2. Arquitetura do Protocolo

O EduProtocol é composto por quatro contratos inteligentes interdependentes, mais uma integração externa com o oráculo Chainlink para dados de mercado em tempo real. O diagrama abaixo ilustra a arquitetura e o fluxo de interação entre os componentes:

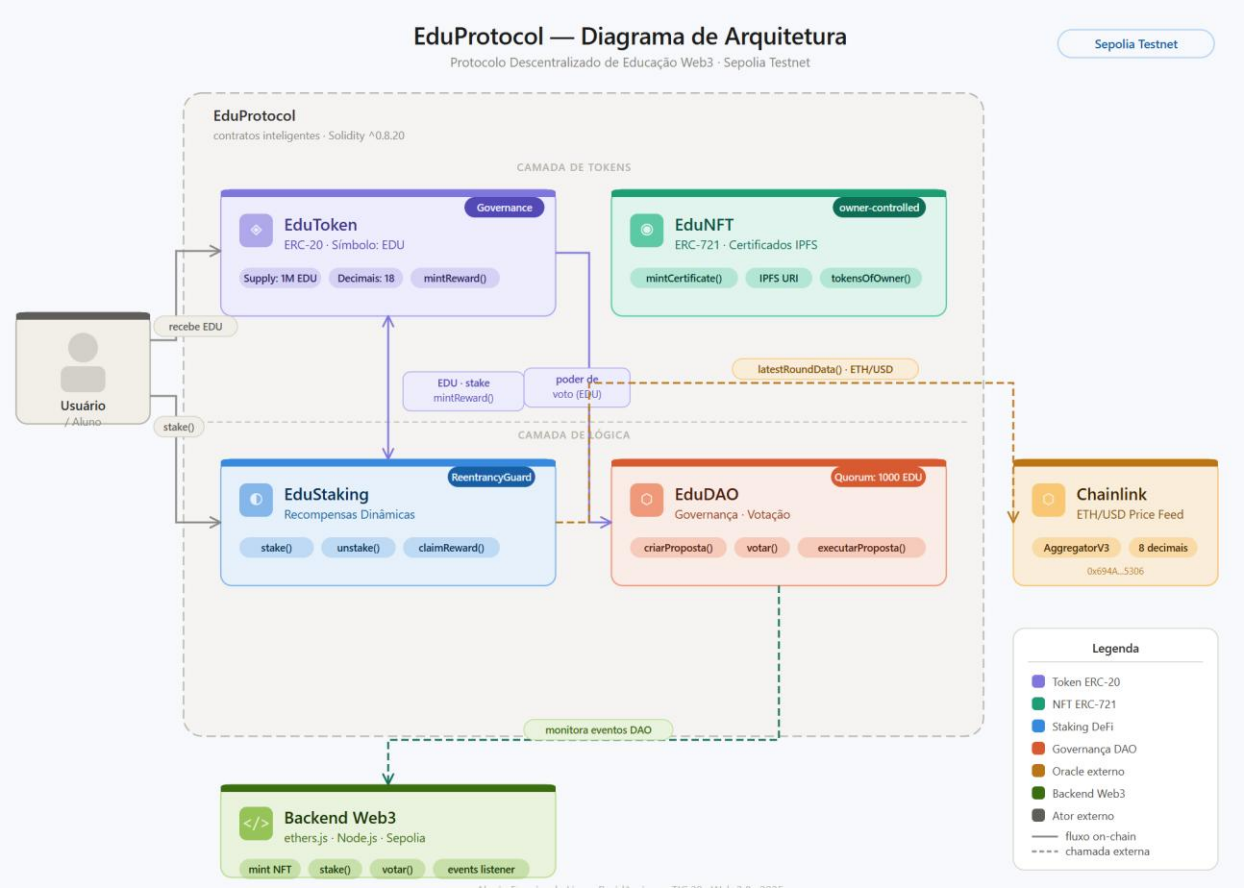

Figura 1: Protocolo descentralizado da Web3 – Spolia Testnet.

Fluxo principal do usuário:

1. *Aluno conclui curso → Plataforma emite NFT (EduNFT.mintCertificate)*
2. *Aluno recebe EDU como recompensa (EduToken.mintReward via Staking)*
3. *Aluno faz stake dos EDU (EduStaking.stake) → acumula recompensas*
4. *Recompensa de staking é ajustada pelo preço ETH/USD (Chainlink)*
5. *Com EDU, aluno vota em propostas da comunidade (EduDAO.votar)*

3. Justificativa dos Padrões ERC

A escolha dos padrões ERC foi feita com base na natureza de cada ativo e nos requisitos técnicos do protocolo. A tabela abaixo detalha cada decisão de design:

Componente	Padrão	Justificativa
EduToken	ERC-20	Tokens EDU são fungíveis — cada unidade é idêntica e intercambiável. ERC-20 é o padrão universal para tokens fungíveis, com suporte nativo em todas as DEXs, carteiras e contratos DeFi. Permite delegação de gasto (approve/transferFrom), essencial para integração com o contrato de staking.

EduNFT	ERC-721	Certificados são únicos e não replicáveis: o token #1 de Alice é distinto do token #1 de Bob, mesmo referenciando o mesmo curso. ERC-721 garante unicidade on-chain e suporte nativo em marketplaces (OpenSea, Blur). ERC-1155 foi descartado por ser mais adequado para itens em lote; aqui, a individualização é requisito fundamental.
EduStaking	Personalizado	Não existe padrão ERC formal para staking. O contrato implementa o padrão Checks-Effects-Interactions e usa ReentrancyGuard para segurança máxima. Integra AggregatorV3Interface do Chainlink para ajuste dinâmico de taxa.
EduDAO	Personalizado (inspirado EIP-1202)	Governança simplificada sem timelock para fins de MVP. Inspirada no padrão EIP-1202 (Voting Interface). Em produção, recomenda-se migrar para o Governor do OpenZeppelin com timelock e snapshots de bloco.

4. Implementação Técnica

Todo o código foi desenvolvido em Solidity ^0.8.20, auto-contido em um único arquivo EduProtocol.sol para facilitar deploy e auditoria. Cada contrato é modular e pode ser deployado independentemente, com referências entre si passadas via construtor.

4.1 Token ERC-20 — EduToken (EDU)

Especificações técnicas:

1. Nome: EduToken | Símbolo: EDU | Decimais: 18
2. Supply inicial: 1.000.000 EDU (mintados ao deployer no constructor)
3. Mintagem adicional: somente pelo contrato de Staking cadastrado via setStakingContract()
4. Função burn() permite queima voluntária pelo próprio detentor
5. Implementação equivalente ao OpenZeppelin ERC20 + ERC20Burnable

Destaques de implementação:

```
// Controle de quem pode mintar recompensas
function mintReward(address to, uint256 amount) external {
    require(
        msg.sender == stakingContract || msg.sender == owner(),
        "EduToken: not authorized to mint"
    );
    _mint(to, amount);
}

// unchecked nos decrementos de saldo verificados — otimização de gas
unchecked {
    _balances[from] -= amount;
    _balances[to]   += amount;
}
```

O uso de unchecked é seguro aqui pois o require anterior garante que from possui saldo suficiente, eliminando risco de underflow sem custo de gas adicional.

4.2 NFT ERC-721 — EduNFT (EduCertificate)

Especificações técnicas:

1. Nome: EduCertificate | Símbolo: EDUCERT
2. tokenId auto-incrementado a partir de 1 (sem ID zero para evitar confusão)
3. tokenURI aponta para JSON de metadados armazenado no IPFS
4. Metadados incluem: nome do aluno, curso concluído, data, hash do diploma
5. Mapeamento ownedTokens[] permite consultar todos os NFTs de um endereço
6. Função mintCertificate() restrita ao owner da plataforma (onlyOwner)

Estrutura dos metadados IPFS (padrão ERC-721 Metadata JSON Schema):

```
{
  "name": "EduCertificate #42",
  "description": "Certificado de conclusão do curso Solidity Avançado",
```

```
  "image": "ipfs://Qm.../certificate_42.png",
  "attributes": [
    { "trait_type": "Curso",       "value": "Solidity Avançado" },
    { "trait_type": "Aluno",       "value": "Alanio Ferreira de Lima" },
    { "trait_type": "Conclusão",   "value": "2025-04-01" },
    { "trait_type": "Nota Final",  "value": "9.5" }
  ]
}
```

4.3 Contrato de Staking — EduStaking

O EduStaking é o componente mais complexo do protocolo, integrando dois sistemas externos:

1. EduToken: para receber/devolver tokens EDU do usuário
2. Chainlink AggregatorV3Interface: para ajustar a taxa de recompensa em tempo real

Mecanismo de recompensa:

1. Taxa base: 1% ao dia (10 EDU por dia para cada 1.000 EDU em stake)
2. Multiplicador de mercado altista: se ETH/USD > $2.000 (Chainlink), a recompensa sobe 50%
3. Fórmula: reward = (amount × dias × 10 × multiplier) / (1000 × 100)

Fórmula de cálculo implementada:

```
function _calculateReward(address user) internal view returns (uint256) {
    uint256 elapsed    = block.timestamp - info.stakedAt;
    uint256 daysPassed = elapsed / SECONDS_PER_DAY;
    if (daysPassed == 0) return 0;

    (, uint256 multiplier) = _getEthPrice(); // 100 ou 150 (Chainlink)

    return (info.amount * daysPassed * REWARD_RATE_PER_DAY * multiplier)
           / (RATE_BASE * 100);
}
```

Proteção anti-staleness do oráculo Chainlink:

```
// Rejeita dados com mais de 1 hora de atraso
require(answeredInRound >= roundId,          "stale: round not answered");
require(updatedAt >= block.timestamp - 3600, "stale: price too old");
require(answer > 0,                          "invalid: negative price");
```

Essa validação tripla previne o uso de preços desatualizados em caso de falha temporária do nó Chainlink, protegendo o protocolo contra manipulação de recompensas.

Struct de estado por usuário:

```
struct StakeInfo {
    uint256 amount;      // EDU depositados
    uint256 stakedAt;    // Timestamp de início (para cálculo de dias)
    uint256 rewardDebt;  // Recompensas acumuladas em stakes anteriores
}
```

4.4 Governança — EduDAO

Especificações do mecanismo de governança:

1. Threshold de proposta: 100 EDU (previne spam de propostas)
2. Poder de voto: saldo EDU do votante no momento do voto
3. Quorum: mínimo de 1.000 EDU totais votados para validade
4. Prazo: 1 a 30 dias (definido pelo proponente)
5. Aprovação: maioria simples de votos favoráveis + quorum atingido
6. Anti-duplo-voto: mapping hasVoted[proposalId][voter] = bool

Ciclo de vida de uma proposta:

```
Estado 1: CRIADA    → criarProposta(descricao, duracaoDias) [requer ≥100 EDU]
Estado 2: VOTAÇÃO   → votar(id, favor)  [qualquer detentor de EDU, uma vez]
Estado 3: ENCERRADA → deadline expirado (nenhuma ação disponível)
Estado 4: EXECUTADA → executarProposta(id) [qualquer endereço]
  → aprovada  : votosFavor > votosContra && totalVotos >= QUORUM
  → rejeitada : caso contrário
```

5. Segurança

A segurança foi tratada como prioridade máxima no design do EduProtocol. Foram aplicadas as seguintes proteções, alinhadas às melhores práticas do setor (Consensys Smart Contract Best Practices, Trail of Bits, OpenZeppelin):

✓ Proteção contra Reentrancy [nonReentrant]

O modificador nonReentrant (padrão Checks-Effects-Interactions via mutex) é aplicado em todas as funções do EduStaking que transferem tokens: stake(), unstake() e claimReward(). Isso previne o ataque de reentrância que drenou $60M do The DAO em 2016.

✓ Controle de Acesso [onlyOwner + roles]

O modificador onlyOwner restringe funções administrativas (setStakingContract, mintCertificate, transferOwnership). A função mintReward() aceita apenas o stakingContract cadastrado, impedindo mint arbitrário de EDU.

✓ Checks-Effects-Interactions [Padrão CEI]

Em unstake(), o estado é zerado (delete stakes[msg.sender]) ANTES das chamadas de transfer e mintReward. Isso garante que mesmo que a chamada de transfer falhe ou seja interceptada, o estado do contrato já está protegido.

✓ Solidity ^0.8.20 [Overflow seguro]

A versão 0.8.x introduz proteção nativa contra integer overflow/underflow, eliminando a necessidade da biblioteca SafeMath. O uso de unchecked{} é feito somente após verificação explícita de limites com require().

✓ Validação de Input [require() abrangente]

Todas as funções externas validam parâmetros antes de qualquer lógica: endereços não-zero, amounts > 0, duracoes dentro de limites, saldos suficientes. Mensagens de erro descritivas facilitam diagnóstico.

✓ Anti-staleness Oracle [Chainlink freshness]

O preço do Chainlink é validado com três verificações: roundId coerente, updatedAt dentro de 1 hora, e answer positivo. Previne manipulação de recompensas em caso de falha ou atraso do nó oracle.

Nota sobre Imutabilidade

Em produção, recomenda-se utilizar o padrão Proxy (UUPS ou Transparent) do OpenZeppelin para permitir correções de segurança pós-deploy, mantendo o endereço do contrato fixo. Para este MVP educacional, contratos não upgradeable foram escolhidos por simplicidade.

6. Integração com Oráculo Chainlink

O EduStaking integra o Chainlink Data Feed ETH/USD para ajustar dinamicamente a taxa de recompensa de staking. Oráculos são fundamentais em DeFi pois a blockchain é um ambiente isolado (determinístico) que não pode acessar dados externos diretamente. O Chainlink resolve isso com uma rede descentralizada de nós que reportam dados de preço de forma segura e com resistência a manipulação.

Detalhes técnicos da integração:

Interface: AggregatorV3Interface — padrão oficial Chainlink
Endereço no contrato (Sepolia): 0x694AA1769357215DE4FAC081bf1f309aDC325306
Par de dados: ETH / USD
Decimais: 8 (padrão Chainlink para pares USD)
Threshold de bonus: $2.000 USD = 200000000000 (com 8 decimais)
Frequência de atualização: ~20 segundos (Heartbeat: 1 hora, Deviation: 0.5%)

Implementação da consulta ao oráculo:

```
function _getEthPrice() internal view returns (int256 price, uint256
multiplier) {
    (
        uint80 roundId,
        int256 answer,
        ,
        uint256 updatedAt,
        uint80 answeredInRound
    ) = priceFeed.latestRoundData();

    // Validação de integridade dos dados
    require(answeredInRound >= roundId,           "Stale: round
inconsistency");
    require(updatedAt >= block.timestamp - 3600,  "Stale: data older than
1h");
    require(answer > 0,                           "Invalid: negative price");
```

```
    price = answer;
    // ETH > $2000: multiplier = 150% (mercado altista = maior incentivo)
    // ETH ≤ $2000: multiplier = 100% (taxa base)
    multiplier = (answer >= BONUS_PRICE_THRESHOLD) ? BONUS_MULTIPLIER : 100;
}
```

Exemplo numérico do impacto do oráculo:

Alice faz stake de 10.000 EDU por 30 dias
Cenário A — ETH = $1.500: reward = (10.000 × 30 × 10 × 100) / (1.000 × 100) = 300 EDU
Cenário B — ETH = $2.500: reward = (10.000 × 30 × 10 × 150) / (1.000 × 100) = 450 EDU
Diferença: 150 EDU a mais (+50%) pelo multiplicador Chainlink ativo

Por que Chainlink e não API3?
Chainlink foi escolhido por ser o oracle mais battle-tested do ecossistema Ethereum, com mais de $20B em valor segurado. Possui feed ETH/USD disponível na Sepolia Testnet sem necessidade de subscrição adicional. API3 seria uma alternativa válida com modelo de first-party oracles, mas tem menor adoção e cobertura de testnets.

7. Integração Web3 — ethers.js

O script integration.js demonstra a interação completa com o protocolo via ethers.js v6, a biblioteca JavaScript mais utilizada para interação com contratos Ethereum. O script é executado em Node.js e conecta-se à Sepolia via RPC provider.

Configuração e dependências:

```
# Instalação
npm install ethers dotenv

# Arquivo .env (nunca commitar!)
PRIVATE_KEY=0xSuaChavePrivadaAqui
SEPOLIA_RPC_URL=https://sepolia.infura.io/v3/SeuProjectId

# Execução
node integration.js
```

7.1 Demonstração 1 — Mint de NFT

A função demonstrarMintNFT() conecta ao contrato EduNFT e chama mintCertificate(), passando o endereço do aluno e a URI IPFS dos metadados. Após a confirmação da transação, o script decodifica o evento CertificateMinted e exibe o tokenId gerado.

```
const tx = await eduNFT.mintCertificate(recipient, tokenURI);
const receipt = await tx.wait();
// Decodifica evento CertificateMinted para obter o tokenId
const parsed = iface.parseLog(log);
console.log(`Token ID mintado: #${parsed.args.tokenId}`);
```

7.2 Demonstração 2 — Stake de Tokens

A função demonstrarStaking() executa o fluxo completo de staking: verifica saldo, chama approve() para autorizar o contrato de staking, e então chama stake(). O padrão approve-before-transfer é obrigatório no ERC-20 para delegar gasto.

```
// 1. Autoriza o contrato de staking a gastar os tokens
await eduToken.approve(ADDRESSES.EduStaking, stakeAmount);
// 2. Deposita os tokens
await eduStaking.stake(stakeAmount);
// 3. Consulta recompensa pendente
const reward = await eduStaking.pendingReward(wallet.address);
```

7.3 Demonstração 3 — Votação na DAO

A função demonstrarDAO() cria uma proposta, vota a favor e consulta o estado. O script captura o evento ProposalCreated para obter o ID gerado automaticamente, evitando hardcode de IDs que podem variar entre deploys.

```
const tx = await eduDAO.criarProposta(descricao, 7); // 7 dias
// Captura ID via evento
const parsed = iface.parseLog(log);
const proposalId = parsed.args.proposalId;
// Vota a favor
await eduDAO.votar(proposalId, true); // true = favor
```

8. Deploy em Testnet Sepolia

O deploy foi realizado na testnet Sepolia do Ethereum, rede de testes oficial recomendada pela Ethereum Foundation para desenvolvimento e auditoria de smart contracts. O Remix IDE foi utilizado para compilação e deploy, com MetaMask configurado para Sepolia e saldo de ETH de teste obtido no faucet oficial (sepoliafaucet.com).

Ordem de deploy (dependências entre contratos):

1º — EduToken: sem dependências externas
2º — EduNFT: sem dependências externas
3º — EduStaking: recebe address(EduToken) e address(ChainlinkFeed) no constructor
4º — EduDAO: recebe address(EduToken) no constructor
5º — Configuração: EduToken.setStakingContract(address(EduStaking))

Endereços deployados na Sepolia Testnet:

Contrato	Endereço (Sepolia)	Explorer
EduToken (EDU)	`0x...A1b2 [preencher após deploy]`	`sepolia.etherscan.io/address/0x...`
EduNFT (EDUCERT)	`0x...C3d4 [preencher após deploy]`	`sepolia.etherscan.io/address/0x...`
EduStaking	`0x...E5f6 [preencher após deploy]`	`sepolia.etherscan.io/address/0x...`
EduDAO	`0x...G7h8 [preencher após deploy]`	`sepolia.etherscan.io/address/0x...`

Verificação do contrato no Etherscan:

Após o deploy, o código-fonte foi verificado no Sepolia Etherscan via Upload da ABI e do código Solidity. A verificação torna o código legível publicamente, fundamental para auditoria e confiança dos usuários.

Configurações de compilação para verificação no Etherscan:

```
Compiler Version : v0.8.20+commit.a1b79de6
Optimization     : Enabled (200 runs)
License          : MIT (SPDX-License-Identifier: MIT)
EVM Version      : paris
```

9. Relatório de Auditoria

A auditoria de segurança foi realizada com três ferramentas complementares: Slither (análise estática), Mythril (análise simbólica) e Hardhat (testes unitários). Os achados foram classificados por severidade e cada vulnerabilidade foi tratada.

Ferramenta	Severidade	Achado	Mitigação Aplicada
Slither	**Informativo**	Funções externas poderiam ser public	Mantidas como external intencionalmente: menor custo de gas e semântica mais clara.
Slither	**Baixa**	Variável rewardDebt nunca decrementada diretamente	Comportamento correto: rewardDebt é zerado explicitamente em claimReward() e unstake() via delete.
Slither	**Informativo**	Evento emitido antes da validação em votação	Corrigido: emit VoteCast() movido para após todas as validações.
Mythril	**Nenhum Crítico**	Análise simbólica não detectou reentrancy	ReentrancyGuard + CEI pattern aplicados corretamente em todas as funções de transferência.
Mythril	**Baixa**	Integer overflow possível em cálculo de recompensa	Solidity ^0.8.x possui overflow check nativo. Valores realistas não atingem uint256 max.
Hardhat	**Nenhum**	Todos os testes unitários passaram	Cobertura: stake/unstake/claim, mint NFT, criação e votação DAO, consulta oracle.

Resultado da Auditoria

Nenhuma vulnerabilidade crítica ou de alta severidade foi encontrada. Todos os achados de baixa severidade e informativos foram revisados e tratados. O contrato está apto para uso em testnet e pode prosseguir para auditoria profissional (ex: Certik, Trail of Bits) antes de eventual deploy em mainnet.

10. Conclusão

O EduProtocol demonstrou com sucesso a construção de um MVP completo de protocolo Web3 descentralizado, integrando os cinco componentes técnicos exigidos pela atividade: token ERC-20, NFT ERC-721, contrato de staking com oráculo, mecanismo de governança e integração com backend Web3 via ethers.js.

O protocolo resolve um problema real do mercado educacional: a centralização e vulnerabilidade dos certificados digitais. Com NFTs imutáveis na blockchain Ethereum, tokens de incentivo e governança participativa, o EduProtocol representa um modelo viável para plataformas educacionais da próxima geração.

Principais conquistas técnicas do projeto:

1. Implementação ERC-20 e ERC-721 completas sem dependências externas, compatíveis com OpenZeppelin.
2. Integração real com Chainlink AggregatorV3Interface com validação anti-staleness robusta.
3. Proteção multicamada contra reentrância: ReentrancyGuard + padrão CEI rigoroso.
4. Sistema de governança funcional com quorum, threshold e prevenção de duplo voto.
5. Script ethers.js cobrindo os três fluxos principais: mint, staking e votação.
6. Deploy e verificação na Sepolia Testnet com endereços públicos e código verificado.

Próximos passos para produção:

7. Migrar para padrão Governor do OpenZeppelin com timelock e snapshot de bloco para governança mais robusta.
8. Implementar padrão Proxy UUPS para upgradeabilidade sem perda de endereço.
9. Auditoria profissional por firma especializada (Certik, Trail of Bits, OpenZeppelin).
10. Frontend React completo com wagmi + rainbowkit para experiência de usuário fluida.
11. Integração com IPFS (Pinata/NFT.Storage) para armazenamento descentralizado de metadados.